Budget-viable commitment for finite-language LLM completion

Ya Bai, GuoZhen Bian*, Cheng Chang, ZhengGuang Cui, HaiLong Qin, ZhiHui Huang, Yue Wang

China Telecom Cloud Network Operating System R&D Center, Beijing, China

* Correspondence: biangz@chinatelecom.cn

Version 8.8, 27 October 2026. Research draft with completed editorial and formal revision. New-method task experiments and author declarations remain incomplete; this version is not submission-ready.

Abstract

An answer-boundary stop can return a locally complete value inside an unfinished artifact, while a grammar-legal continuation can exhaust its token budget before acceptance. We formulate budget-viable commitment as a controller for append-only completion over a declared token language. At activation, selected values are bound and a finite completion language is verified against a consumer contract. The controller retains only tokens with an accepting continuation inside the remaining budget, then commits a validated artifact or records an explicit abstention. We characterize this support by backward reachability and prove conditional completion after entry into a viable state. These properties concern structural acceptance and field preservation, rather than answer truth, and do not establish

algorithmic priority over equivalent grammar controllers. A finite-trie implementation integrates with Transformers generation. Controlled CPU checks cover distance pruning, boundary cases, binding, model support and failure accounting. Historical stopping data provide motivation: 204 validity repairs contained 114 correct and 90 incorrect answers, showing that validity and correctness can diverge. A format-only reduction further shows that deterministic shortest completion can match acceptance and bound values without additional model calls. Real-model comparisons on task-relevant unbound fields, matched grammar baselines, multiple budgets and complete request costs remain necessary. No task-quality improvement or GPU speed advantage for the proposed controller has yet been measured.

Keywords: constrained decoding; finite languages; token budgets; verified completion; abstention

1. Introduction

Completing an answer field and delivering an accepted artifact are different events. A numerical answer may be locally complete while a surrounding wrapper remains open. Conversely, a structurally permitted continuation may still need more tokens than a request has available. Returning early, constraining legal prefixes and enforcing a completion deadline address related but distinct problems.

We study **budget-viable commitment** over a declared finite token language. A provisional draft is either accepted directly or used to bind values and compile verified suffixes. During completion, the controller excludes next tokens that leave no accepting path within the remaining budget. A final contract check determines commitment. Unsupported, infeasible and failed attempts remain observable abstentions; their work is not omitted from evaluation.

The current implementation is append-only. It can complete a supported prefix, but cannot undo an earlier answer, edit selected preceding fields or recover a missed binding event through rollback. The proposal therefore concerns a specific completion controller,

rather than a general document-repair system. Binding preserves content selected by a rule; it does not establish that content is true.

The present contributions are a precise separation of acceptance, budget feasibility and answer correctness; a finite-language implementation of the resulting controller with controlled software checks; and a retrospective motivation analysis with a specified evaluation protocol. The backward-reachability recurrence itself is not claimed as a novel algorithm. Whether the controller has an independent methodological advantage over existing structured generation remains unestablished.

Three questions organize the study. RQ1 asks whether the implemented controller satisfies its conditional structural and binding invariants. RQ2 asks whether budget pruning improves completion and correct-commit yield over matched grammar-only continuation. RQ3 asks whether model-scored completion provides value beyond deterministic repair at an acceptable total cost. Controlled checks address RQ1 on finite examples; RQ2 and RQ3 still require the experiments specified in Section 6. Historical stopping results in Section 5 do not evaluate the new controller.

2. Related work and comparison boundary

Lexically constrained decoding and grammar-constrained generation are established approaches (Hokamp and Liu, 2017; Geng et al., 2023). Practical libraries such as XGrammar and Outlines support structured generation (XGrammar project, 2026; Outlines project, 2026), and native Transformers stopping criteria support token-boundary return decisions (Hugging Face, 2025). These capabilities are relevant baselines, rather than evidence that the present controller is distinct.

Budget-aware reasoning addresses how much inference to allocate (Wang et al., 2024; Han et al., 2025; Muennighoff et al., 2025). Our stated budget is the number of additional output tokens before acceptance in a declared controller language. This differs from estimating reasoning sufficiency: the controller can ensure a permitted structural ending without ensuring that the answer is correct.

An implementation with the same token transitions, acceptance set, distance filter and scoring rule can be algorithmically equivalent to ours. Renaming that implementation or adding a submission certificate does not establish a new decoding paradigm. The verified XGrammar documentation also identifies its XGrammar-2 successor as dynamic structured generation; comparison cannot be restricted to old static-grammar examples. The documentation was checked, but this draft does not claim a completed full-paper survey or an empirical comparison with that system.

The present literature boundary is consequently limited. The scope must be checked against deadline-aware constrained search, structured editing and verified generation before asserting priority. No claim is made that these fields lack the same recurrence. A useful systems or task contribution would require measured value beyond equivalent grammar control and deterministic completion.

3. Formal controller

3.1. Consumer contract, provisional prefix and binding

Let x be the prompt and y the generated output prefix. A declared contract $\mathcal{C}$ has a final acceptance predicate $V_{\mathcal{C}}(y)$ and a task extractor g. A binding rule identifies values b to preserve. Define $P_b(y)$ as their preservation predicate. Neither $V_{\mathcal{C}}$ nor P_b is a gold-answer oracle. Correctness, when a reference exists, is evaluated separately through g.

At an activation prefix, compilation constructs a token controller with partial transition $\delta(s, v)$ and accepting states F. An accepting path must decode to an artifact satisfying both $V_{\mathcal{C}}$ and P_b. This is a soundness requirement for the declared predicates, not a claim to implement all Markdown, LaTeX or JSON semantics. The current wrapper validator is a limited operational heuristic. An independent final call to the predicate rechecks each committed artifact; it is not an independently developed semantic oracle.

The protocol has four persistent states: DRAFT, REPAIR, COMMIT and ABSTAIN. BIND is a transition operation that fixes values and constructs the completion controller,

rather than a separately persisted state in the supplied code. Already accepted prefixes commit directly. The current interfaces append tokens to the entire frozen prefix; they do not implement selective rewriting of earlier unbound text.

3.2. Budget and token language

Let B be the total allowed number of generated output tokens and let t have already been generated. The repair allowance is $r = B - t$. In a supplied-prefix experiment, the allowance instead refers only to new completion tokens, and the supplied prefix's cost must be reported separately. Prompt tokens do not count toward this output cap, but their processing time remains part of a declared request measurement.

Each transition consumes one model token. Acceptance triggers an immediate return action without requiring an additional EOS token. EOS is forbidden inside a nonaccepting repair. Any emitted EOS during drafting still counts as a generated token. If a different implementation requires EOS to complete an artifact, that transition and its cost must be included in its controller.

For state s, define the distance within the declared controller language:

$$d(s) = \min\{|z| : \delta^*(s, z) \in F\},$$

with $d(s) = \infty$ for unreachable acceptance and $d(s) = 0$ at an accepting state. For $s \notin F$ and $r > 0$, define

$$\mathcal{A}_r(s) = \{v : \delta(s, v) \text{ is defined, } d(\delta(s, v)) \leq r - 1\}.$$

The distance is measured in model tokens, not characters or bytes. Exactness is relative to this controller: a finite suffix list with selected tokenizations can omit other valid strings and other tokenizations of the same string. Its distance is not a global minimum over all artifacts accepted by the consumer.

3.3. Model scores and viable support

For positive normalizer Z, the locally masked distribution is

$$q_r(v \mid x, y, s) = \frac{p_\theta(v \mid x, y)\mathbf{1}\{v \in \mathcal{A}_r(s)\}}{Z}, \qquad Z = \sum_u p_\theta(u \mid x, y)\mathbf{1}\{u \in \mathcal{A}_r(s)\}.$$

The implementation uses greedy selection among permitted finite scores. Negative infinity may represent zero model support; it is harmless if another permitted token has a finite score. An all-negative-infinity support, NaN or positive infinity on permitted entries produces an explicit failure. No assertion is made that local masking samples from the model conditioned on eventual global acceptance.

3.4. Reachability characterization and conditional completion

Proposition 1 — feasible first-token characterization. For a nonaccepting state and remaining budget $r > 0$, a legal token v belongs to $\mathcal{A}_r(s)$ if and only if some accepting continuation of at most r tokens begins with v.

Proof. If v is retained, the next state has an accepting suffix of length at most $r - 1$, which can follow v. Conversely, remove the first token from any accepting path of length at most r; its suffix witnesses $d(\delta(s, v)) \leq r - 1$.

Thus $\mathcal{A}_r$ is maximally permissive with respect to existential structural feasibility in this language. This characterization is ordinary backward reachability; it is not a priority claim. Structural feasibility does not ensure that the retained path has positive model probability or a correct answer.

Proposition 2 — nonempty support and budget-bounded acceptance. Suppose repair begins at s_0 with $d(s_0) \leq r_0$. Assume exact transitions and distances, a permitted token with positive model support at every nonaccepting step, selection restricted to the viable support, and successful execution without an external failure. Then structural acceptance occurs in at most r_0 new tokens.

Proof. A shortest path from any nonaccepting state with $d(s) \leq r$ supplies an allowed first token, so the structural support is nonempty. After any allowed emission, decrementing r preserves $d(s) \leq r$. At zero remaining tokens, this inequality implies $d(s) = 0$, hence acceptance. The model-support and execution assumptions ensure the process does not abort earlier. Immediate return at acceptance ends the process without another token.

Proposition 3 — certified output. If all reachable accepting states decode to artifacts satisfying $V_{\mathcal{C}}$ and P_b, and commitment revalidates those predicates, every committed artifact satisfies them. This is a contract-relative postcondition. It provides no factual or task-correctness guarantee and cannot establish soundness of the validator itself.

The statements are conditional on reaching a viable activation point. They do not guarantee a candidate, supported compilation or successful commitment for every request. An optimistic lower bound on d is insufficient: it can retain a path that misses the deadline. These results also concern a token budget, not a bound on wall-clock latency.

3.5. Equivalence, deterministic reduction and a quality counterexample

Under the same state, language, distance oracle, scores, remaining budget, acceptance rule and tie-breaking, another grammar engine applying $\mathcal{A}_r$ has the same token support and greedy trajectory. If every grammar-legal next token already satisfies the bound, the distance mask is inactive. Such matched cases are consistency controls rather than opportunities for an output-quality advantage.

Proposition 4 — format-only reduction. Suppose a viable compiled repair language contains only formatting variants of the same bound complete answer. A deterministic shortest accepting path preserves that answer and satisfies the same contract without a new model call. Its inserted suffix uses no more tokens than any other accepting path in that language.

Proof. Select a shortest accepting path of length $d(s)$. The compiler's postconditions establish acceptance and preservation. Selecting and serializing a path requires no

probability-model query.

This reduction applies directly to the wrapper prototype. Deterministic insertions still have CPU and serialization costs, and the common prefix retains its model-generation cost. The result does not compare total latency without measurement, nor does it apply when choosing an unbound field changes task correctness.

A two-branch counterexample shows why structural feasibility cannot replace quality evaluation. Let one accepting path contain token a and encode a wrong answer, while a correct answer requires tokens b, c. With one remaining token, the mask can retain the wrong one-token path and exclude the correct two-token path. Commitment remains structurally valid. Binding can likewise exclude a correct answer before repair begins. Therefore neither correct-commit improvement nor noninferiority follows from Propositions 1–3 or from historical zero-loss observations.

4. Reference implementation and verification

4.1. Operational activation rule

The draft-to-wrapper interface uses a total cap B and a fixed reserve h, with $0 \leq h < B$. After each draft token, it checks the following rules in order:

1. If the current artifact satisfies the closure-balanced reference predicate, commit it.
2. If natural EOS has occurred without acceptance, abstain.
3. If the strict marker predicate identifies a complete candidate, bind that value and propose supported closing suffixes. Verify their decoded context, final acceptance and candidate preservation. Enter repair only when a compiled accepting suffix fits $B - t$; otherwise abstain.
4. If no candidate was found and $t \geq B - h$, abstain; otherwise continue drafting.

Candidate recognition takes precedence over the no-candidate deadline at the same step, while the total cap still applies. The reserve is a drafting policy, not an oracle guaranteeing a candidate by the deadline. Unsupported candidate states currently abstain immediately

rather than continuing to search for a later candidate. This behavior must be held fixed or explicitly ablated in experiments.

The supplied-prefix interface starts directly from a finite language and an acceptance/binding predicate pair. An already accepted prefix commits with zero new tokens and zero model calls, including at zero remaining budget. Other prefixes require a viable nonempty completion. The caller supplies the prefix and suffix language without a gold-answer argument. This API separation alone cannot certify that a caller's predicates are gold-independent; benchmark construction must be audited separately.

4.2. Compilation, complexity and scope

Each proposed suffix is tokenized and decoded after the actual output prefix; paths failing exact context round-trip, final acceptance or preservation are rejected. The verified token paths form a trie. If their total length is M, the trie has at most $M + 1$ nodes. Building it is $O(M)$; a postorder distance calculation uses $O(N + E)$ time and memory for N nodes and E edges. These bounds exclude tokenization, candidate enumeration and predicate execution. Enumeration may be impractical for a large compositional answer space.

The current implementation traverses the emitted repair prefix when retrieving a trie state and constructs a dense vocabulary mask. At repair step j, this adds prefix traversal and outgoing-edge inspection plus $O(|\mathcal{V}|)$ mask construction, before accounting for the model, decoding or validation. Python callbacks and score checks may synchronize host and device. No near-zero overhead claim follows from the linear distance preprocessing bound.

The implementation is batch-size-one greedy generation with no KV rollback, speculative path, arbitrary JSON Schema compiler or general document grammar. Two interfaces demonstrate wrapper closure and finite structured completion, respectively. The latter allows selection of an unbound field from supplied alternatives, but is not itself a dataset benchmark or a general grammar compiler.

4.3. Controlled software checks

The revised code passed 22 controlled CPU tests using PyTorch 2.6 and Transformers 4.48.2 with a small random GPT2 and scripted scores. Checks include finite-trie distance and pruning invariants; candidate and identifier preservation; choice of an unbound label; unsupported syntax and context mismatch; invalid EOS and drafting deadlines; deterministic repair; accepting and nonaccepting zero-budget cases; zero model support and invalid scores; and retention of trace and cost after final-certificate or runtime failure.

The new edge-case checks repair accounting and integration behavior rather than measuring answer quality. Both generation adapters return explicit constraint/runtime failure records when handled, retain available generated tokens and remove their model hooks. Forward counts record entered calls, including calls that raise an error; they do not count only successful returns. A benchmark wrapper must still capture configuration, tokenization, predicate, device and infrastructure exceptions outside these handled paths. These tests establish no pretrained-model accuracy, practical compilation coverage or CUDA speedup.

5. Historical motivation and evidence limits

The historical stopping experiments used Qwen2.5-7B-Instruct-1M (Yang et al., 2025) and Mistral-7B-Instruct-v0.3 (Mistral AI, 2024) on GSM8K (Cobbe et al., 2021) and ARC-Challenge (Clark et al., 2018). There were 600 main questions per task, reused across models. Uniform ran to EOS or a 512-token cap. Complete returned the earliest certified terminal answer line. The full initial 25,200-call protocol and all ten historical table bodies remain in Supplement A–B; the later timing study added 11,200 scored calls and 480 warmups. Repeated calls and sessions are not independent new questions.

The frozen quality contract required a last line-start `####` value with a newline or EOS certificate and no later non-whitespace content. It did not validate the surrounding document. It differs from both the prototype's closure-balanced heuristic and a future typed-record contract. All paired scores in Table 1 use the frozen historical contract.

Table 1. Historical Uniform-to-Complete validity repairs, not results of constrained completion. Each cell uses 600 paired questions. Counts are model–question outcomes.

Model / task	Invalid to valid	Correct repairs	Wrong repairs
Qwen / GSM8K	28	14	14
Qwen / ARC-C	0	0	0
Mistral / GSM8K	129	81	48
Mistral / ARC-C	47	19	28

Across 2,400 model–question pairs, 204 outputs changed from invalid to valid: 114 then matched the reference and 90 did not. There were no observed validity losses, and the shortened-output set exactly matched the validity-repair set. These are descriptive paired findings on reused questions, not 2,400 independent question identities. They locate savings in contract-breaking continuation, without showing improved reasoning.

The interpretation changes under alternative contracts. Last-marked-answer extraction gave zero Complete-minus-Uniform accuracy difference in all four cells, while the closure-balanced heuristic produced a negative difference for Qwen/GSM8K. These results and all unfavorable outcomes are retained in Supplement B. New comparisons must use the same final contract across policies rather than mixing these endpoints.

Historical isolated Shadow-minus-Complete monitoring overhead ranged from 16.46 to 51.18 ms. Complete was not universally faster than Uniform. Follow-up concurrent timing used different measurement boundaries and is not pooled with the isolated run. Its available summary package lacks per-call raw follow-up timing records needed to reconstruct the intervals independently. These measurements motivate complete request accounting but do not measure the new controller's cost.

6. Evaluation protocol awaiting execution

6.1. Tracks and task construction

A matched-prefix track will isolate completion control by giving methods the same prefix, bound values, token language and remaining budget. A separate end-to-end track will include prompt processing, drafting, failed activation, compilation, completion, validation and abstention. Supplied-prefix timing cannot support end-to-end speed or coverage claims.

The task set must distinguish a format-only reduction control from structured prediction with task-relevant unbound fields. The former compares deterministic closure with model-scored closure without presuming a semantic benefit. The latter can bind a public instance identifier while leaving a categorical or numeric answer to predict. ARC candidates may include all visible options. A numeric answer language must be defined independently of gold answers; a manually supplied reference-answer list is invalid. The present wrapper interface cannot execute a general numeric-grammar task without additional compiler and runner development.

Historical cohorts have been inspected and can support a declared known-cohort re-evaluation. They cannot be relabeled unseen holdouts. Independent instances or tasks are needed for broader generalization claims. Input IDs, contracts, compiler capabilities and binding rules will be frozen before final evaluation. Failed compilation remains in the denominator for every frozen eligible request.

6.2. Comparators and ablations

The matched-prefix comparisons require deterministic shortest completion, model-scored grammar-only continuation, the full budget mask and an independently implemented version of the same distance recurrence. Shared language and acceptance checks are essential. The equivalent recurrence checks consistency; output superiority is not expected under identical assumptions. Finite-answer tasks also require a strong simple alternative,

such as development-frozen prior selection or candidate-sequence scoring with all model calls accounted for.

End-to-end comparisons require Uniform, early return on the same target contract, deterministic repair, selective grammar-only repair, the full controller and a practical grammar engine operating from the start. Native string stopping is sufficient competition only when it enforces the declared contract. From-start and selective grammar differ in more than the budget filter and must be interpreted as system comparisons. Historical Complete under its old contract remains a diagnostic, not a primary new-contract comparator.

Ablations isolate removal of the distance filter at the same binding point, selective versus from-start constraints, and complete-answer versus public-structure binding. Any comparison requiring earlier-field editing first requires a new implementation. A deliberately removed validator is a fault-injection check rather than a competitive baseline.

6.3. Budgets, endpoints and failure reporting

Development data will determine tight, intermediate and relaxed total token caps and the reserve rule before testing. Shared-prefix cases include budgets below, at and above the compiled minimum distance; unequal-length accepting branches; inactive masks; and infeasible states. Report how often pruning actually excludes a grammar-legal token. A fixed single cap cannot establish a budget-response curve.

For request i, let D_i indicate a declared submission rather than abstention, $A_i = D_i\mathbf{1}\{V_{\mathcal{C}}(y_i) \wedge P_b(y_i)\}$ indicate a certified submission, and $Q_i = A_i\mathbf{1}\{g(y_i) = g_i^*\}$ indicate a correct certified submission. The primary endpoint is $\sum_i Q_i/n$ over all frozen eligible requests. Certified coverage is $\sum_i A_i/n$, and conditional accuracy is $\sum_i Q_i/\sum_i A_i$, undefined when no output is certified. Invalid submissions, $\sum_i D_i(1 - A_i)$, and binding violations remain separately visible. For a baseline without controller states, a normal returned output is a declared submission, then scored by the same predicates. Invalid raw returns are retained rather than silently relabeled or discarded. A schema check that rejects outputs can change declared submission counts without improving the all-request

quality endpoint.

Also report budget overruns and failure categories: unsupported compilation, context mismatch, no candidate, infeasible distance, invalid EOS, constraint failures and runtime errors. Infrastructure failures need a prespecified retry rule and their own labels; no failed request may silently disappear. No ratio with a zero denominator is assigned an invented zero or perfect value.

Length-dependent pruning may favor short wrong answers. Include canonical short-label controls, answer-length strata, erroneous bindings and offline reference-reachability analysis. Reference information is available only to the scorer, never to inference, budget selection or post hoc sample exclusion. Supported-language or reference-reachable subsets are diagnostics rather than replacements for the all-request endpoint.

6.4. Cost, statistics and reproducibility

Record model-generated, deterministically inserted and returned token IDs separately, together with forward calls and costs of compilation, masking, validation and failed attempts. Measure synchronized isolated request latency with matched model precision, attention backend, batch size and hardware; randomize or balance policy order. Separate cold compilation from a declared reusable-cache regime. Report mean and P50/P95 latency and paired uncertainty. Concurrent throughput would require a distinct experiment, not extrapolation from request times.

A pilot will support sample-size planning from a prespecified meaningful correct-commit difference and paired discordance; timing planning uses paired variability. Repeated calls, budgets and model answers to the same question are not independent questions. Question-level pairing and clustering must preserve these dependencies. A primary setting or multiplicity adjustment must be fixed before confirmatory cross-budget and cross-model claims. All cells and unfavorable results remain reportable.

The archive will contain frozen manifests, version and environment hashes, full token traces, binding and distance events, statuses/errors, request-time boundaries and executable

scoring/analysis. An audited separation of controller inputs from reference labels is required. No sample size, main comparison or favorable outcome has yet been inferred from an unexecuted experiment.

7. Discussion

The controller adds a remaining-budget condition to structural prefix legality and makes acceptance a distinct return event. Its conditional guarantee is useful for specifying behavior, but follows from reachability over a declared language. It cannot alone support a paradigm claim relative to an equivalent grammar implementation. A larger experimental advantage must be attributed to a documented difference in language, activation, scoring, compiler cost or task behavior, rather than a name.

The current finite language limits both coverage and scalability. Missing valid paths can cause unnecessary abstention, and an exhaustive language over rich fields can be expensive to construct. The wrapper contract is a heuristic rather than a full document parser. Append-only binding may freeze an incorrect answer and cannot use later correction. Validity is consequently a necessary interface property, not a proxy for reasoning quality.

The deterministic reduction is a substantive boundary. If empirical gains are restricted to formatting-equivalent suffixes, the appropriate contribution is verified interface repair. Task-relevant unbound fields may justify model-scored continuation, but they also expose quality loss through budget pruning and require strong non-generative or candidate-scoring baselines. Those comparisons remain unexecuted.

No task-level inference improvement or latency advantage is established by the current evidence. The historical studies support their own stopping and monitoring conclusions; the CPU tests support implementation behavior on controlled finite cases. New model runs and source-level comparison with relevant prior systems are required before a stronger contribution claim can be assessed.

8. Conclusions

Budget-viable commitment provides an explicit finite-language specification for append-only constrained completion and validated return. Exact token-level reachability gives conditional structural completion inside a remaining budget, while final acceptance checks preserve declared contract and binding predicates. These properties do not imply true answers, global compiler completeness, successful activation for every request or wall-clock speed. The finite-trie implementation and controlled checks make the specification testable. Historical evidence motivates separating validity from correctness, and the format-only reduction identifies a necessary deterministic comparator. Real-model task and cost comparisons remain necessary before this draft can support an empirical algorithm claim.

Data and code availability

The revision package contains controller code, two Transformers interfaces, deterministic repair, controlled tests, historical analysis inputs/summaries and the ten-table historical supplement. The full old raw experiment archive is a separate prerequisite for rerunning the historical JSONL analysis. The raw follow-up timing archive and a complete task-level benchmark runner for the new controller remain unavailable in this package. Documentation snapshots are versioned and hashed. No deposited permanent archive DOI or final publication release is asserted.

Declarations pending author verification

Funding, competing interests, author contributions and final author approval remain unconfirmed. The supplied author order, affiliation and correspondence address are retained for verification. These entries must be replaced with factual, journal-compliant declarations before submission; absence of funding or conflicts is not assumed.

AI assistance was used in preparing this revised text and developing/checking the supplied code. The authors must verify and complete the disclosure against the actual research

workflow and the selected journal's policy. All reported endpoints are computational; no human-rating result is claimed. Human authors remain responsible for all content and final approval.

References

Clark P, Cowhey I, Etzioni O, Khot T, Sabharwal A, Schoenick C, Tafjord O. 2018. Think you have solved question answering? Try ARC, the AI2 Reasoning Challenge. arXiv:1803.05457. https://arxiv.org/abs/1803.05457

Cobbe K, Kosaraju V, Bavarian M, Chen M, Jun H, Kaiser L, Plappert M, Tworek J, Hilton J, Nakano R, Hesse C, Schulman J. 2021. Training verifiers to solve math word problems. arXiv:2110.14168. https://arxiv.org/abs/2110.14168

Geng S, Josifoski M, Peyrard M, West R. 2023. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. Proceedings of EMNLP 2023, 10932–10952. DOI: 10.18653/v1/2023.emnlp-main.674. https://aclanthology.org/2023.emnlp-main.674/

Han T, Wang Z, Fang C, Zhao S, Ma S, Chen Z. 2025. Token-budget-aware LLM reasoning. Findings of ACL 2025, 24842–24855. DOI: 10.18653/v1/2025.findings-acl.1274. https://aclanthology.org/2025.findings-acl.1274/

Hokamp C, Liu Q. 2017. Lexically Constrained Decoding for Sequence Generation Using Grid Beam Search. Proceedings of ACL 2017 (Volume 1: Long Papers), 1535–1546. DOI: 10.18653/v1/P17-1141. https://aclanthology.org/P17-1141/

Hugging Face. 2025. Transformers v4.48.2: generation stopping criteria, including StopStringCriteria. Versioned implementation. https://github.com/huggingface/transformers/blob/v4.48.2/src/transformers/generation/stopping_criteria.py (accessed 26 October 2026).

Mistral AI. 2024. Mistral-7B-Instruct-v0.3 model card. https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3

Muennighoff N, Yang Z, Shi W, Li XL, Fei-Fei L, Hajishirzi H, Zettlemoyer L, Liang P, Candes E, Hashimoto T. 2025. s1: Simple test-time scaling. arXiv:2501.19393v3. https://arxiv.org/abs/2501.19393v3

Outlines project. 2026. Official project README, commit `c52af8472c6fe8a607a733ec11b709476af77a96`. https://github.com/dottxt-ai/outlines/blob/c52af8472c6fe8a607a733ec11b709476af77a96/README.md (accessed 27 October 2026).

Wang J, Jain S, Zhang D, Ray B, Kumar V, Athiwaratkun B. 2024. Reasoning in token economies: Budget-aware evaluation of LLM reasoning strategies. Proceedings of EMNLP 2024, 19916–19939. DOI: 10.18653/v1/2024.emnlp-main.1112. https://aclanthology.org/2024.emnlp-main.1112/

XGrammar project. 2026. Official project README, including the XGrammar-2 announcement, commit `e085851aec8f417924be5f460d1e038cac0e3bd5`. https://github.com/mlc-ai/xgrammar/blob/e085851aec8f417924be5f460d1e038cac0e3bd5/README.md (accessed 27 October 2026).

Yang A, Yu B, Li C, Liu D, Huang F, Huang H, Jiang J, Tu J, Zhang J, Zhou J, Lin J, et al. 2025. Qwen2.5-1M technical report. arXiv:2501.15383. https://arxiv.org/abs/2501.15383